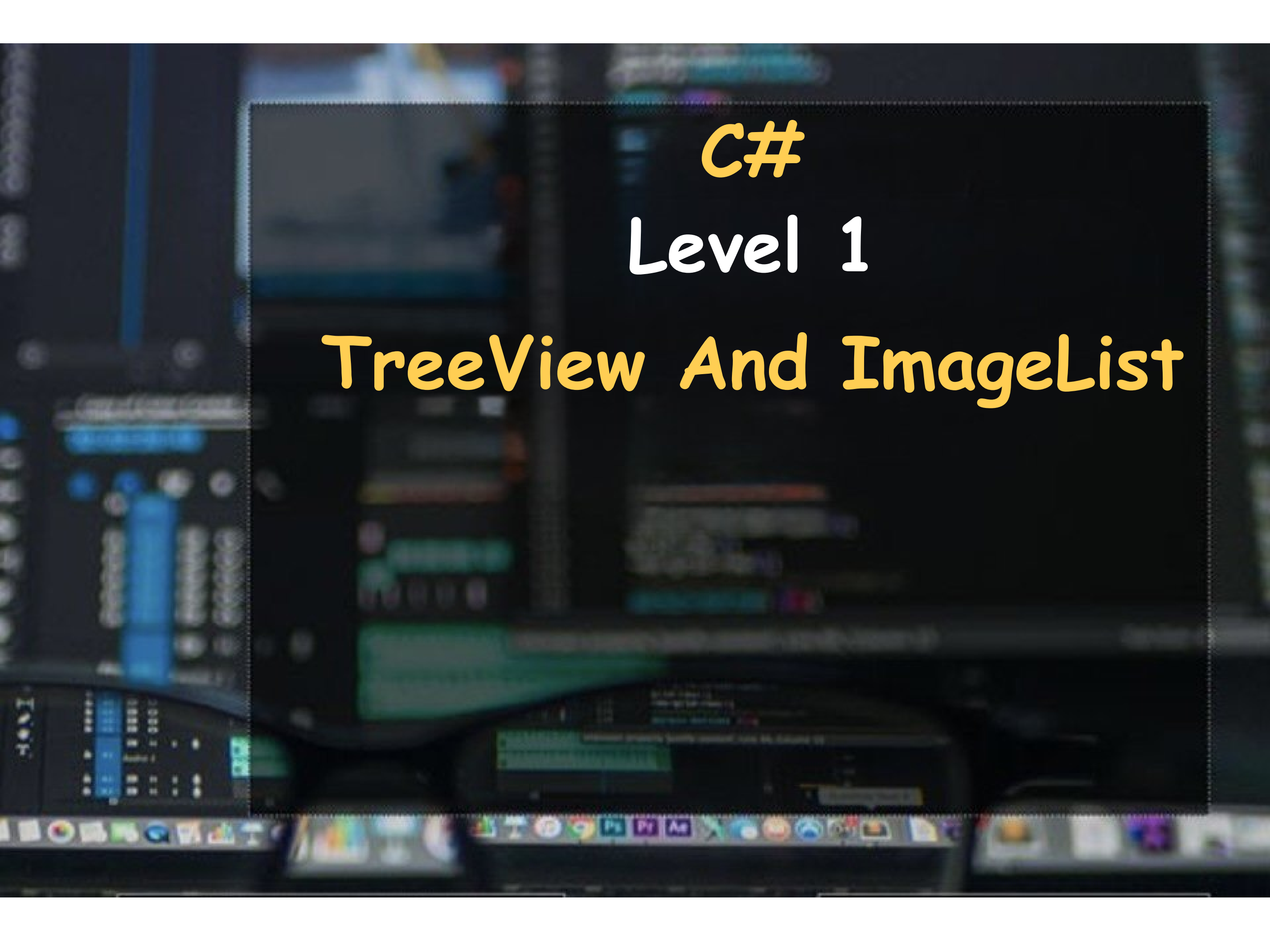


The background is a blurred image of a computer monitor. It shows a dark-themed code editor with various lines of code in different colors (blue, green, white). On the left side, there's a sidebar with a tree view. At the bottom, a Windows taskbar is visible with several application icons like Chrome, Photoshop, and others.

*C#*

Level 1

*TreeView And ImageList*



C#

Level 1

# TreeView And ImageList

# Definition

## TreeView

 Windows Forms control used to display hierarchical data using TreeNode objects.

 Nodes can contain parent and child nodes at multiple levels.

## ImageList

 Component that stores a collection of images by index.

 Can be connected to TreeView to display images next to nodes.

# Explanation

## Node Selection

-  ``SelectedNode`` gets the currently selected `TreeNode`.
-  ``SelectedNode.Text`` gets the text displayed by that node.

## Checking Nodes

-  ``AfterCheck`` occurs when a node's ``Checked`` state changes.
-  ``e.Node.Checked`` determines whether the node is checked or unchecked.
-  ``CheckTreeNode()`` applies the same state to all child nodes.

## Recursive Processing

-  The method loops through ``node.Nodes`` and changes each child's ``Checked`` state.
-  If a child contains more nodes, the method calls itself recursively to process deeper levels.

## Procedure

1. Add a TreeView to the Windows Form.
2. Create parent and child TreeNodes.
3. Handle `MouseDownClick` to get the selected node.
4. Display the selected node's text using `SelectedNode.Text`.
5. Handle `AfterCheck` when a node's checkbox changes.
6. Pass the node and its checked state to `CheckTreeViewNode()`.
7. Recursively apply the state to all child nodes.
8. Connect an ImageList if node images are required.

## 💡 Example

```
private void treeView1_MouseDoubleClick(object sender, MouseEventArgs e)
{
    MessageBox.Show(treeView1.SelectedNode.Text);
}
private void treeView1_AfterCheck(object sender, TreeViewEventArgs e)
{
    CheckTreeViewNode(e.Node, e.Node.Checked);
}
private void CheckTreeViewNode(TreeNode node, bool isChecked)
{
    foreach (TreeNode nNode in node.Nodes)
    {
        nNode.Checked = isChecked;

        if (nNode.Nodes.Count > 0)
        {
            CheckTreeViewNode(nNode, isChecked);
        }
    }
}
```

## Note

- ◆ `TreeNode` represents one item in the TreeView.
- ◆ `node.Nodes` contains its child nodes.
- ◆ `SelectedNode` gets the selected node.
- ◆ `Checked` is a Boolean property.
- ◆ `ImageIndex` selects the normal node image.
- ◆ `SelectedImageIndex` selects the image when the node is selected.
- ◆ Recursion is used to process nodes at all hierarchy levels.



# Comparison

## TreeView

- ✓ Displays hierarchical data.
- ✓ Uses TreeNode objects.
- ✓ Supports parent-child relationships.

## ImageList

- ✓ Stores images by index.
- ✓ Can be assigned to TreeView.
- ✓ Displays icons next to nodes.

## ⊘ Warning

- ◆ `SelectedNode` can be `null` when no node is selected.
- ◆ Changing `Checked` inside `AfterCheck` can trigger additional `AfterCheck` events.
- ◆ Image indexes must exist in the assigned ImageList.

# ✦ Summary

- ✓ TreeView displays hierarchical information using TreeNodes.
- ✓ `SelectedNode` → gets the selected node.
- ✓ `Checked` → gets or sets the checkbox state.
- ✓ `AfterCheck` → handles changes to the checked state.
- ✓ `ImageList` → stores images for TreeView nodes.
- ✓ `ImageIndex` / `SelectedImageIndex` → control node images.
- ✓ Recursion → processes all child levels.

Thank you 😊